

操作系统

一、计算机系统、操作系统概述

CPU、memory、I/O 设备由系统总线连接

CPU 寄存器 { 用户可见
控制、状态

中断: Interrupt Handler (操作系统的一部分)

存储: 层次结构 (registers → cache → main memory)

I/O 设备 { 控制器 (由设备驱动程序使用)
设备本身

系统启动: 主板上的 BIOS 启动, 检测、初始化硬件

↓
读入、执行 boot block

↓
找到 secondary boot block 并读入、执行
(加载操作系统)

↓
操作系统从 BIOS 中读取信息

↓
用户空间初始化

操作系统: 用来管理硬件、控制程序、充当二者接口的程序
{ 虚拟机 (一种抽象)
资源管理器

内核: 在内存中的 OS 有最常使用的功能

系统调用: OS 和用户程序间的接口

OS 提供的抽象 { 进程、线程
地址空间、内存管理
文件

2. 进程: OS 进行资源分配和调度度的基本单位



进程定义: 一个正在执行的程序

特性: 动态性、并发性、独立性、异步性

每个进程都认为自己独占机器 (OS 提供接口抽象)

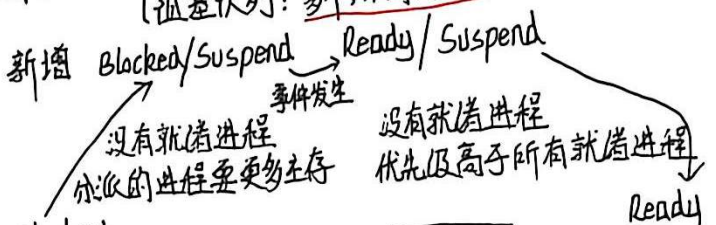
指令轨迹: 表征每个进程的行为, 指令序列



New: 为进程关联标识符、分配并构建管理进程的表格

Exit: OS 临时保存进程相关的信息

排队模型 { 就绪队列: 多个, 优先级
阻塞队列: 多个, 不同 Event



Blocked OS 维护一个 process table

PID	PCB

→ 特定进程的属性集合

PCB 管理 { 用 PCB 中的链接字
构造索引表

进程映像: 程序、数据、栈、属性的集合

PCB { 进程标识
处理器状态
进程控制

进程内存
stack 临时数据
↓
heap 动态分配
data 全局变量
text 代码

进程创建: 分配唯一的进程标识符
为进程分配空间
初始化 PCB
建立链接
创建、扩展其他数据结构

程序 → 进程: 加载代码、数据 → 分配堆栈 → I/O → 启动

执行模式 { 用户态 (应用程序)
内核态 (启动、仅执行 OS 代码)

切换到内核态 { 中断 (外部)
陷入 (错误、异常)
系统调用 (显式)

上下文切换: 存储当前状态, 读另一个进/线程状态并执行

进程切换: 保存当前上下文、更新 PCB、移到合适队列

↓
选择另一进程, 更新 PCB、内存管理数据结构
加载新上下文

三、线程：CPU调度和执行的基本单位

线程包含：线程ID、程序计数器、寄存器组、栈

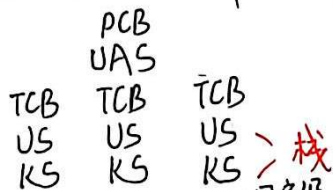
线程与同一进程中的其他线程共享：代码、数据、堆、其他资源

线程栈内包含帧（已调用未返回过程）：局部变量、返回地址

线程切换的代价更低（仅涉及寄存器和栈指针切换）

多线程好处：响应性、资源共享、经济性、可伸缩性

多线程进程模型：



调度和分派基于线程完成

每个进程有自己的线程表来追踪线程属性

用户级线程ULT即所有线程管理由应用程序完成 (Threads library) (内核不知)

阻塞会阻塞所有线程 (Jacketing 技术)

先内核级线程KLT：内核完成所有管理 (TB、PB内核中)

UT和KT可以多对一、一对一、多对多(复用)

互斥、并发：互斥与同步

进程交互：1. 竞争 $\Rightarrow$ 临界资源：不可同时被多个进程访问

2. 共享 $\Rightarrow$ 确保数据完整性 与互斥

3. 通信 $\Rightarrow$ 消息传递

对临界区问题解决：强制互斥、闲时让进、有限等待

解决互斥的方法：

1. 软件方法

① Dekker算法 (用于两进程互斥)

flag、turn表示谁可以访问

```
例：P0: { while(true) {
    flag[0] = true; // 意图进入
    while(!flag[1]) { // 判断，想不想
        if (turn == 1) { // 若想，看能不能
            flag[0] = false;
            while(turn == 1) { // 等待
                flag[0] = true;
            }
        }
    }
}
```

简化资源逻辑

② Peterson算法

```
例：P0: { while(true) {
    flag[0] = true;
    turn = 1;
    while(!flag[1] && turn == 1) {
        // 等待
    }
}
```

2. 硬件方法

① 禁用中断：强制顺序交错 (有风险)

② 机器指令

Test and Set: TSL R_x, lock / 会锁定内存总线

if (i == 0) { i = 1; return true; } else return false;

while (!testandset(&bolt)) { ... }

Exchange: 原子地交换两个位置的内容

```
while(true) {
    key[i] = 1;
    while(key[i] != 0) { // 为0临界区空
        exchange(key[i], bolt); // 忙等待
    }
    exchange(key[i], bolt);
}
```

3. 信号量：一种特殊变量进行信号

① 普通信号量 count、queue

wait: count --; if (count < 0) 阻塞、挂起

signal: count ++; if (count <= 0) 解除阻塞 (累加)

② 二元信号量 value、queue (0,1)

wait: if (value == 1) value = 0; else 阻塞

signal: if (queue 为空) value = 1; else 解除一个阻塞

强信号量：阻塞时间长的先释放 (弱，不指定顺序)

生产者/消费者问题：

无限缓冲区：两个信号量 { N: 产品数, S: 访问缓冲区 }

有限缓冲区：循环，加一个信号量 e 代表空格数

wait, signal: 原子实现

理发店问题：公平理发店，谁先来谁先被服务

读者/写者问题：

① 读者优先：出现读者时抢写信号量，无读者释放

② 写者优先：出现写者时抢读信号量，可读时，第一个读者抢写信号量

4. 管程：一个软件模块 { - / 多个过程, - 一个初始化序列, 局部数据 (只能由管程访问) }

进程 (最多一个)

5. 消息传递：send receive

五、死锁：2个以上进程并发时，相互持有对方所需的资源并等待对方释放

必要条件：互斥、占有且等待、非抢占、循环等待

可重用资源
可消费资源

资源分配图：资源 R_1 持有 $\rightarrow$ 进程 P 请求 $\rightarrow$ 资源 R_2

$\hookrightarrow$ 内可有多个实例
 $\hookrightarrow$ 无环：无死锁
 $\hookrightarrow$ 有环：可能有死锁

处理死锁：

1. 鸵鸟算法

2. 预防：限制，让四个条件之一无效

3. 避免：动态决定当前资源分配请求是否会造成死锁

① 进程启动拒绝 $\rightarrow$ 总量

Resource = $\langle R_1 \dots R_m \rangle$ Available = $\langle V_1 \dots V_m \rangle$ $\rightarrow$ 可用

Claim = $\begin{pmatrix} C_{11} & \dots & C_{1m} \\ \vdots & & \vdots \\ C_{n1} & \dots & C_{nm} \end{pmatrix}$ Allocation = $\begin{pmatrix} A_{11} & \dots & A_{1m} \\ \vdots & & \vdots \\ A_{n1} & \dots & A_{nm} \end{pmatrix}$
n个进程 需求 已分配

P_{n+1} 仅在： $R_i \geq C_{n+1,i} + \sum_{k=1}^n C_{ki}$ $\forall i$ 成立时启动

② 资源分配拒绝（银行家算法）

对于进程的请求，判断批准后的状态是否安全 $\rightarrow$ 可执行完

4. 检测与恢复：用请求矩阵 Q

Mark 分配矩阵中全为 0 的进程（不占资源）
之后找未 Mark 的进程 i ：若 $Q_i \leq W$ ，则可运行

Mark 此进程， $W += i$ 的分配资源（ i 释放），重复

出现意外：有死锁

恢复：抢占资源、逐个中止、回滚重启……

哲学家就餐问题：只允许 4 个人用餐

六、单处理器调度：决定哪个进程被运行

三级调度：

创建 $\xrightarrow{\text{长程调度}}$ 就绪 $\xrightarrow{\text{短程调度}}$ 运行
(批处理队列) $\swarrow$ $\searrow$
就绪/挂起 阻塞/挂起

长程调度：决定哪些程序被插入进入系统进行处理

中程调度（交换功能的一部分）

短程调度：从就绪进程中选一个，将 CPU 分配给他

进程行为：交替进行计算脉冲和 I/O 请求

调度：
非抢占式
抢占式
环境：
批处理
交互式
实时

短程调度策略：

1. FCFS 先来先服务：选择 ready 中最早的进程

归一化周转时间： $\frac{\text{周转时间}}{\text{服务时间}}$ (等待+服务) $\uparrow$ 则队列 $\downarrow$

偏爱受 CPU 限制的进程，最简单

2. RR 轮转：固定时间片（单次最长运行时间）

不公平：受 I/O 限制每次用很短 CPU

$\downarrow$
VRR，引入 FCFS 辅助队列，接收就绪 I/O 阻塞的进程
有更高优先级

3. SPN (SJF) 最短进程优先（非抢占）

较长进程可能饥饿 所有 Job 同时可用为非抢占最长

需估计每个进程的处理时间

估计 $S_{n+1} < \frac{\alpha T_n + (1-\alpha) S_n}{\alpha T_n + (1-\alpha) S_n}$

4. SRT 最短剩余时间（SPN 的抢占版本）：更优周转时间

5. HRRN 最高响应比优先（非抢占）

$\hookrightarrow R = \frac{\text{等待} + \text{处理}}{\text{处理}}$ (多次计算选择)

多级队列调度（队列间固定优先级抢占式）

就绪队列被分为几个有不同优先级、进程类型的队列
(进程固定进入某一个队列，每个队列有自己的调度算法)

多级反馈队列

动态优先级：每次从执行变就绪，进入更低优先级队列

其他：FCFS

最低优先级：RR

公平份额调度：彩票调度、步长调度 $\rightarrow$ 大值彩票数

七、内存管理

逻辑地址 { 编译和装入时绑定：相同
物理地址 { 执行时绑定：不同

内存抽象：地址空间 进程用虚拟地址内存（每个进程独立）

动态地址转换：用户 VA $\rightarrow$ MMU $\rightarrow$ Main Memory

连续内存分配：

内存分区：

① 固定分区：将可用内存划分为具有固定边界的区域

程序过大
程序太小（内部碎片）：不等大小分区 $\Rightarrow$ 放置 { 队列 } 最小可
算法 { 分区 } (用分区)

② 动态分区

进程被分配恰好大小的内存 $\Rightarrow$ 外部碎片（紧凑）

分配算法：
首次适配：从开始
最佳适配：全局
邻近适配：从上次

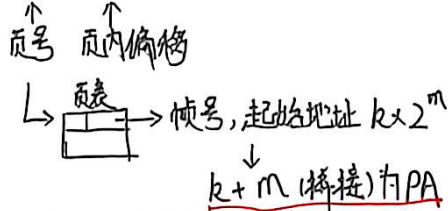
③ 伙伴系统： $2^{n-1} < S \leq 2^n$ ，原则平分；同级空间合并

非连续内存分配

分页: 直接等分物理内存(帧)和逻辑内存(页)

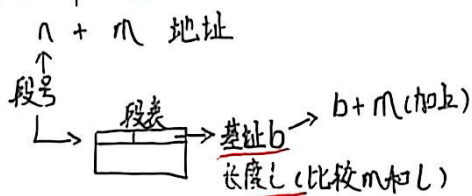
OS 为每一个进程维护一个页表, 显示页的帧位置

地址转换: $n + m$ 位地址



保护位: 页的权限 页表项有效/无效位

分段: 用户视角, 可变大小段 (逻辑视角)



段页式: $s + p + o$ 地址

↑ 段号

↓ 段表 → 页表 → 帧号 → 地址

1. 虚拟内存: 用户感知 (大)

thrashing 抖动: 大多时间交换片段而不是执行指令

控制结构:

页表项: 用于索引 控制位

PTE (不包含磁盘地址)

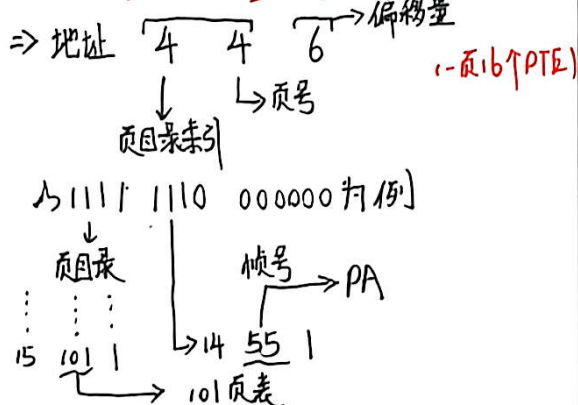
→ present
dirty bit
referenced bit
protection bit

页表结构:

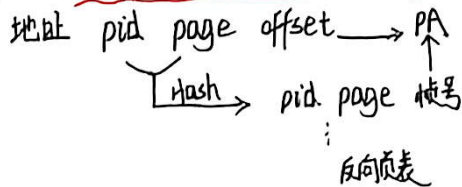
多级页表: 将页表切分为页大小的多个单元 (整页无效直接忽视)

以两级为例: 16KB 的地址空间 页大小为 64B PTE 为 4B

⇒ 256 页 页表为 $256 \times 4B$ ⇒ 页表可分为 16 页 → 4 位



反向页表: 实存中每一页帧有一个条目, 跟踪谁位于其中 负载控制: 挂起某进程



TLB 快表: 页表项的高速缓存 MMU 的一部分 (无旁路页表)

(也有分段、段页式, 前面已有)

交换空间: 磁盘上预留的来回移动页的空间

虚拟内存策略 (最小化缺页发生率)

1. 读取策略: 请求式调页
频调页

2. 放置策略: 决定页在内存中的驻留位置 NUMA

3. 替换策略: 调入新页时, 选择内存中要替换的页

帧锁定: 关联帧定位 不可替换

算法: OPT 最优: 选下次引用时间最长的页 (理想)

先进先出 FIFO: 驻留时间最长的页被替换

Belady 异常: 主存量个性能 ↓ ⇒ 栈算法 m 时为 $m+1$ 的 3 倍

最近最少使用 LRU: 替换最长时间内未被引用的页

链表实现: 引用时放在头部

计数器: 引用时时钟递增, 关联到页

矩阵: k 被引用, k 行全变 1, k 列全变 0, 值最小

⇒ 近似 LRU: 每页 8 位, 定时将引用位移入最高位, 右移

使用位: 页面首次进内存时设 1, 被引用时设 1

第 2 次机会算法: 检查最近页使用位

时钟策略: 循环遍历

使用位为 1 则置 0, 第一个 0 被替换

增强型: 加一个脏位

找 0 0, 失败就找 0 1 (前者 1 → 0), 重复

基于计数: LFU MFU

页缓冲: 换出页写出之前, 所需页被读入一个空闲帧池的帧中

维护一个已修改页的链表, 空闲提前写入

记住空闲帧池每帧中原是哪个页

驻留集: 进程在内存中的页

工作集: 在时刻 t , 最近 n 次使用的页

$w(t, t)$

⇒ 定义了工作集窗口

PPF: 缺页时检查自上次缺页以来的间隔

大于 F: 清 0 为 0 页, 缩

小于 F: 加一个页

VSWS: 采样开始, 所有 0 设 0; 结束 0 全 1。

最长采样 L 若 L 之前发生 Q 次缺页

4. 清除策略: 请求清除 (页缓冲)

M 之前, 等到 M

M 之后, 结束

九、文件管理

磁盘(有时IO)的概念 → 文件的抽象

文件: 存储在辅助存储设备中的相关信息的集合(命名)

低级字节流文件 { 顺序访问
结构化文件 { 随机访问 → 索引

文件目录: 包含有关文件的信息, 提供映射(文件名 → 文件本身)

结构: 单级目录

两级目录: 每个用户有单独目录

树形结构目录: 目录包含一组文件或子目录

{ 绝对路径名
相对...

有向无环图目录: 目录有共享的子目录/文件

硬链接: 创建与文件相关的inode
目录指向inode

符号链接: 文件内容为名链接的文件的路径名

目录实现: 线性表、哈希表

文件保护:

层次结构的权限

域: 一组(对象, 权利)对

{ 所有者
组
全域

存取控制列表: 对象关联列表, 包含域及其权限

权能: 进程关联列表, 包含对象及权限

二级存储管理: 磁盘分区 文件分配的区段大小 { 可变、大、连续块

文件分配方法:

连续分配: 一组连续的块

链式分配: 块 → 块(包含指针) 顺序处理

FAT文件分配表: 每个磁盘块一个条目 存下一块的块号

索引分配: 单个文件所有指针集中到一个索引块

可变大小区段: 局部连续块 长度

空闲空间管理:

位表: 每个块一位 可用为1

链表: 所有空闲块链接到一起

空闲块列表: 保存空闲块号(分组)

链式空闲区: 指针、长度

文件系统一致性:

块一致性 { use 至少一个为1、不超过1
free

文件一致性: 遍历树 检查计数

UNIX中打开 /usr/ast/mbox

固定的根目录inode → 根目录 → usr的inode

mbox ← ast目录 ← ast的inode ← usr的目录

